PES UNIVERSITY
100 feet Ring Road, BSK 3rd Stage, Bengaluru 560085

Department of Computer Science and Engineering
(AI & ML)
B. Tech. CSE - 5th Semester
Aug – Dec 2026
UE23CS351A
DATABASE MANAGEMENT SYSTEMS (DBMS)
PROJECT REPORT on

FaceTrackDB: Face-Recognition Hostel Attendance and Gate Tracking System

Submitted by:

PES1UG24AM111	Harish Ramesh Kumar	5B
PES1UG24AM116	Harshit Vaish	5B

Class of Prof. ***Smitha GR (Professor and Lab Faculty)***

Abstract

Hostels usually track attendance and gate movement with paper registers, which are slow, easy to fake and impossible to query. FaceTrackDB is a hostel attendance and gate-tracking system that recognises a student's face at the gate and at the evening roll call and records attendance, every entry and exit, leave requests and rule violations automatically. The data is split by kind. PostgreSQL 16 holds the records and enforces the rules: 18 tables, 21 CHECK constraints, 26 foreign keys, an exclusion constraint against overlapping leave, 7 triggers, 4 stored procedures, 26 functions and 6 views (one recursive), and six PostgreSQL roles with row-level security. The Qdrant vector database holds the 512-number face embeddings and answers the nearest-neighbour question "which stored face is closest to this one". A Next.js 16 application provides pages and an API for five roles (student, parent, warden, security, admin) with bcrypt passwords, signed session cookies and role checks on every request. Wardens get a dashboard, full-text search and eleven SQL reports built with CTEs, window functions, subqueries and aggregates. The database is built by one command and checked by 232 SQL tests, and the attendance history shown in the reports is generated demo data, labelled as such. Known limits are stated openly: recognition thresholds are uncalibrated, there is no liveness check, and database-level access control covers only some routes; the rest is enforced by the application.

Table of Contents

Sl. No	Topic	Page No.
1	**1. Introduction** a) Problem Statement	3 3
2	**2. User Requirements Specification - Tech Stack**	4
3	**3. E-R Model** a) List of Entities (weak ones labelled) b) Non-simple Attributes c) ER Diagram Snapshot d) ERD from a diagramming tool	5 5 5 6 6
4	**4. Relational Model** a) List of Relations b) Schema Diagram	7 7 8
5	**5. SQL DDL Statements**	9
6	**6. SQL DML Statements**	11
7	**7. NoSQL / Vector Database Commands**	15
8	**8. Results** a) Resulting Tables' Screenshots	16 16

Sl. No	Topic	Page No.
9	**9. Next.js Project Tree Structure**	18
10	**10. Sample Screenshots of Application Code and Demo**	20
11	**11. Evaluation Rubrics - Completion Status**	24

1. Introduction

a) Problem Statement

In most hostels attendance is still taken by hand. A warden calls out names or signs a paper register at the evening roll call, and a security guard notes who leaves and returns in a notebook. This is slow, it is easy to fake (a friend can answer "present" for someone who is out), and it leaves no history that a warden or a parent can query. Late returns, nights spent outside without leave, and missed roll calls are found, if at all, by reading the notebook.

FaceTrackDB replaces the registers with a face scan at the hostel gate and at the evening roll call. The system recognises the student, records attendance and every gate entry and exit, raises rule violations automatically (late return, leaving without approved leave, missed roll call), and lets a parent approve or reject a leave request in the same application. Wardens get a dashboard, full-text search and eleven SQL reports with a presentation mode; students and parents see only their own data.

Why two databases

The data is of two different kinds. Students, hostels, roll calls, gate logs, leave requests and violations are records with keys, rules and transactions, so they live in **PostgreSQL 16**. A face is a vector of 512 numbers, and the one question asked about it is "which stored face is closest to this one?". That is a nearest-neighbour search, so the vectors live in the **Qdrant** vector database. A small table in PostgreSQL (`face_profiles`) links each vector to its student.

Objectives

1. Replace paper registers with automatic, auditable attendance and gate records.
2. Keep the business rules inside the database (constraints, triggers, stored procedures, transactions) so they hold whatever code writes the data.
3. Use a vector database where it fits (similarity search over faces) and a relational database for everything else, and justify the split.
4. Give each of the five roles only the pages and data it needs.

5. Present the data through dashboards, full-text search and eleven reports built from different SQL techniques.

Scope and limits

The project is a working prototype for coursework. The recognition thresholds are starting values that were checked with a few people, not calibrated on a large group, and there is no liveness check (a printed photo is not detected). The attendance history shown in the reports is **simulated**: it was generated by a seeding script so that the reports have a month of activity to show, and the application labels it as generated. Scans made live during the demonstration are real records. Source code: https://github.com/harish-rameshkumar/FaceTrackDB

2. User Requirements Specification - Tech Stack

Functional requirements by role

Role	What the role can do
Student	See own attendance, gate history and violations; request and cancel (pending) leave.
Parent	See wards' leave, history and violations; approve or reject a ward's pending leave once.
Warden	Dashboard, full-text search, review queue for low-confidence scans, roll calls, resolve violations, eleven reports and presentation mode.
Security	Gate scanner for roll call and gate IN/OUT.
Admin	Everything the warden can, plus the gate, student registration and deletion, face enrolment and face samples, "sign out everywhere".

Business rules enforced by the database

- A gate crossing alternates IN and OUT, in time order, at a gate of the student's own hostel.
- An exit at or after 20:00 without approved leave raises **no_leave**; a return at or after 22:30 raises **late_return**; a student who has not scanned when a roll call closes is marked absent and gets **missed_attendance** unless leave covers the day.
- A leave request is decided once, only by a linked parent; approved leaves of one student may not overlap; a present mark is never downgraded.
- Face data is stored only for a student whose consent is recorded at registration.

Non-functional: bcrypt passwords, signed HttpOnly session cookie (8 h), database-backed login rate limit, token revocation, PostgreSQL roles with row-level security for part of the routes, Docker Compose start-up with pinned images, Asia/Kolkata time zone, and a test for every rule (section 11).

Technology stack

Part	Version	Purpose
PostgreSQL	16.15	Records, constraints, triggers, procedures, views, full-text search
Qdrant	1.19.2	Vector database for 512-d face embeddings (Cosine)
Next.js / React / TypeScript	16.4.2 / 19.3.0 / 5.9.3	Pages and JSON API
Python, FastAPI, InsightFace	3.11, 0.115.6, 0.7.3 (buffalo_l)	Face service: SCRFD detector, ArcFace embeddings
pg, bcryptjs, jose, recharts, Tailwind	8.23.1, 3.0.3, 6.2.13, 3.10.1, 4.3.3	Database access, passwords, JWT, charts, styling

Why this stack, and why a vector database

PostgreSQL holds all records and rules because attendance, gate logs, leave and violations need keys, constraints, triggers and transactions: 18 tables, 21 CHECK constraints, 26 foreign keys, a GiST exclusion constraint, 7 triggers, 4 procedures, 26 functions, 6 views and 6 roles with row-level security. **Next.js** serves pages and API from one TypeScript codebase, with every protected route calling one function, `requireRole`. A separate **Python face service** is needed because the face model runs on onnxruntime; it only turns an image into an embedding and stores nothing.

Qdrant is used because the only non-relational question is "which of N stored faces is closest to this one", a nearest-neighbour search. Qdrant builds an HNSW index over the vectors, supports cosine distance and payload filters; a B-tree index cannot answer a similarity query, and a document or key-value store could hold vectors but would need a full scan in application code. We did not benchmark other vector stores (pgvector, Milvus, Chroma), so the choice rests on fit for the task.

3. E-R Model

a) List of Entities (weak ones labelled)

There is no weak entity in the strict textbook sense: every entity has its own surrogate primary key. The entities that come closest, because they cannot exist without another entity, are highlighted in yellow and called existence-dependent. The two associative entities, `student_parent` (Student M:N Parent) and `attendance` (Student M:N Roll call), are described in section 4.

#	Entity (table)	Primary key	Notes
1	Hostel (`hostels`)	hostel_id	Strong. `name` is UNIQUE.
2	User account (`users`)	user_id	Strong. `email` UNIQUE; `role` CHECK in (student, parent, warden, security, admin).
3	Student (`students`)	student_id	Strong. `roll_no` (SRN) UNIQUE, `user_id` UNIQUE.
4	Parent (`parents`)	parent_id	Strong. `user_id` UNIQUE.
5	Gate (`gates`)	gate_id	Strong. UNIQUE (hostel, name).

#	Entity (table)	Primary key	Notes
6	Camera (`cameras`)	camera_id	Existence-dependent on Gate (not weak: own key).
7	Face profile (`face_profiles`)	profile_id	Existence-dependent on Student, cascades on delete (not weak: own key). One row per enrolled image; links to a Qdrant vector.
8	Face event (`face_events`)	event_id	Strong. One row per recognition attempt, any outcome.
9	Roll call (`attendance_sessions`)	session_id	Strong. UNIQUE (hostel, date, start time).
10	Gate log entry (`entry_exit_log`)	log_id	Strong. One row per IN/OUT crossing.
11	Leave request (`leave_requests`)	leave_id	Strong. Exclusion constraint forbids overlapping approved leave.
12	Violation (`violations`)	violation_id	Strong.
13	Consent record (`b_consents`)	student_id	Existence-dependent 1:1 extension of Student (key is the owner's key, cascades). Not a weak entity: it has no partial key of its own.

b) Non-simple Attributes

- **Composite:** none stored as composite columns. The SRN `roll_no` (for example `22CS104`) and `room_no` (`A-201`) carry structure but are single columns.
- **Multivalued:** modelled as separate relations: a student's one or two parents (`student_parent`) and a student's enrolment images (`face_profiles`, five per enrolment).
- **Derived:** where a student is now (view `v_current_presence`), attendance percentage (`v_attendance_summary`), time outside per trip (`v_report_trips`); two generated tsvector columns support full-text search.

c) ER Diagram Snapshot

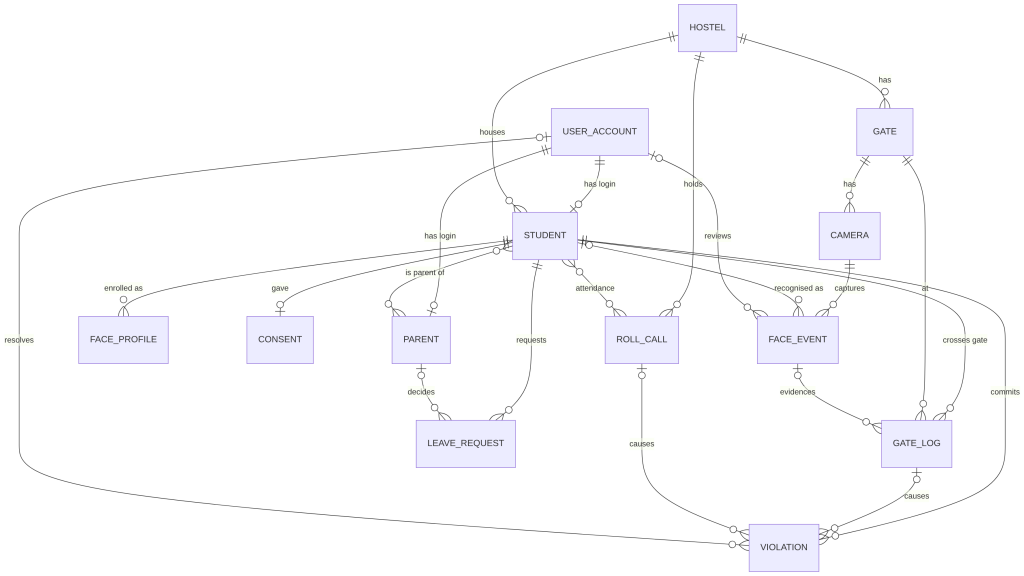


Figure 3.1: Entity-relationship diagram (Mermaid). Associative tables appear as relationships.

d) ERD from a diagramming tool, with relationships and cardinality

The schema diagram (Figure 4.1) is generated from the live database and also exists as an editable draw.io file, `db/docs/schema.drawio`. Main relationships and the mechanism that enforces them:

Relationship	Entities	Cardinality	Enforced by
has login	User - Student / Parent	1 : 0..1	`user_id` NOT NULL UNIQUE; role triggers (FT040)
lives in / has	Hostel - Student / Gate	1 : N	NOT NULL foreign keys
has	Gate - Camera	1 : N	Camera total
is parent of	Student - Parent	M : N	`student_parent`, composite key, CASCADE
enrolled as	Student - Face profile	1 : N (5 per enrolment)	CASCADE; `qdrant_point_id` UNIQUE (vector in Qdrant)
recognised as	Student - Face event	1 : N, partial	NULL student for `no_match`; CHECK
marked in	Student - Roll call	M : N (status)	`attendance`, trigger `trg_attendance_validate`
evidenced by	Attendance / Log - Face event	N : 0..1	Composite FKs `attendance_event_fk`, `log_event_fk`
crossing	Student - Gate log entry	1 : N	trigger `trg_entry_exit_validate`
requests / decided by	Student, Parent - Leave	1 : N	composite FK `leave_decider_is_parent`; `decide_leave`
must not overlap	Leave - Leave	n/a	GiST exclusion `leave_no_overlap`
committed by	Student - Violation	1 : N	CHECK `violations_has_source`; composite FK `violations_log_fk`
consented	Student - Consent record	1 : 1	key is the foreign key; CASCADE

There are no ternary and no recursive relationships. The full list of 27 relationships is in `docs/er-model.md`.

4. Relational Model

a) List of Relations (those that are not entities in the ER model are labelled)

18 base tables and 6 views. Primary keys are underlined; "->" marks a foreign key.

Relation	Attributes	ER entity?
`hostels`	hostel_id, name	Hostel
`users`	user_id, email, password_hash, role, created_at, token_version	User
`students`	student_id, user_id -> users, roll_no, name, hostel_id -> hostels, room_no	Student
`parents`	parent_id, user_id -> users, name, phone	Parent
`student_parent`	(student_id, parent_id) -> students, parents; relation	**No: junction table** (Student M:N Parent)
`gates`, `cameras`	gate_id, hostel_id; camera_id, gate_id, location, status	Gate, Camera
`face_profiles`	profile_id, student_id, qdrant_point_id, enrolled_at	Face profile
`face_events`	event_id, camera_id, student_id, confidence, outcome, event_time, reviewed_by	Face event (audit log)
`attendance_sessions`	session_id, hostel_id, session_date, start_time, end_time	Roll call
`attendance`	(session_id, student_id), event_id, status, marked_at	**No: associative table** (Student M:N Roll call)
`entry_exit_log`	log_id, student_id, gate_id, event_id, direction, event_time	Gate log entry
`leave_requests`	leave_id, student_id, start_date, end_date, reason, status, decided_by, decided_at	Leave request
`violations`	violation_id, student_id, log_id, session_id, type, details, raised_at, status, resolved_by, resolved_at	Violation
`b_consents`	student_id -> students, consented_at, recorded_by	1:1 extension of Student
`login_attempts`	attempt_id, email, ip, succeeded, attempted_at	**No: log table** (login rate limit)
`b_meta`, `b_demo_rows`	key, value; (kind, row_id)	**No: bookkeeping** (demo-data label and tracking)

Foreign keys. There are 26 foreign keys. Four cascade on delete (`student_parent` both sides, `face_profiles`, `b_consents`); the other 22 are NO ACTION, so a student with history cannot be deleted (error FT055). Four composite foreign keys prove ownership: `attendance_event_fk`, `log_event_fk`, `violations_log_fk` and `leave_decider_is_parent`.

b) Schema Diagram

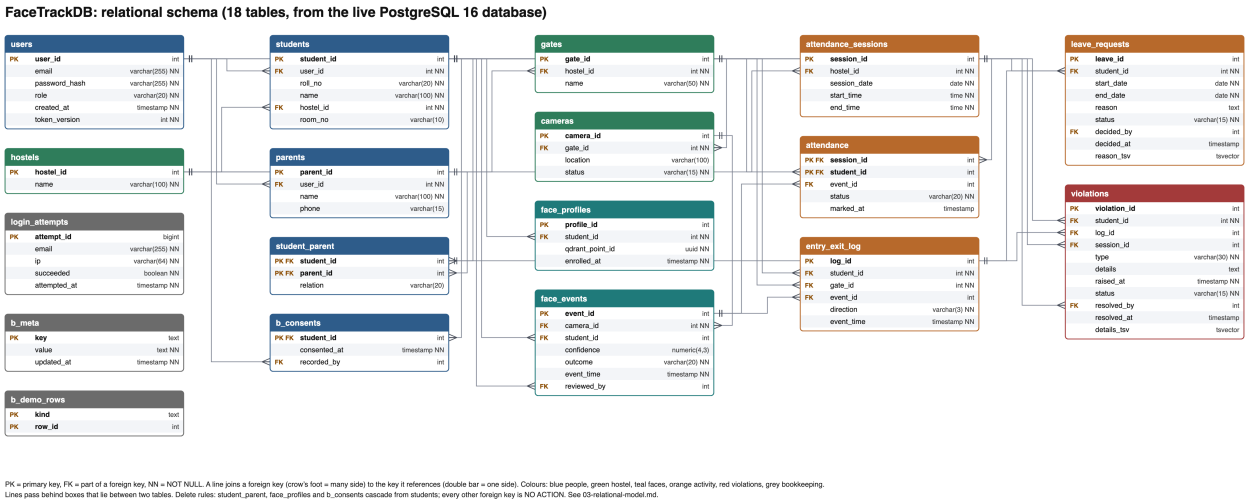


Figure 4.1: Relational schema generated from the live PostgreSQL 16 database (18 tables). PK = primary key, FK = part of a foreign key, NN = NOT NULL.

5. SQL DDL Statements

One command, `./db/load_all.sh`, builds the whole database (schema, seed, triggers, procedures, four migrations, reporting and demo files) in a few seconds on PostgreSQL 16.15. The complete source is in the repository (`db/schema.sql`, 514 lines; `db/triggers.sql`; `db/procedures.sql`; `db/migrations/001-004`). Objects counted from the live catalogue:

Kind	Count	Examples
PRIMARY KEY / FOREIGN KEY	18 (3 composite) / 26 (4 composite)	`attendance_pkey`, `leave_decider_is_parent`
UNIQUE / EXCLUDE (GiST)	10 / 1	`students_roll_no_key`, `leave_no_overlap`
CHECK	21	`users_email_format`, `violations_resolution`
Indexes (partial, GIN)	47 (6, 2)	`idx_face_events_review_queue`, `idx_violations_details_fts`
Views	6 (1 recursive)	`v_absence_runs`, `v_current_presence`
Triggers / procedures / functions	7 / 4 / 26	`trg_entry_exit_validate`, `decide_leave`, `register_student`
Roles / RLS policies	6 / 26 on 8 tables	`ft_student`, `ft_app`; `own_rows`, `staff_all`

Key DDL

```
CREATE TABLE leave_requests (
    leave_id    SERIAL PRIMARY KEY,
    student_id  INT         NOT NULL REFERENCES students(student_id),
    start_date  DATE        NOT NULL,
    end_date    DATE        NOT NULL,
    reason      TEXT,
    status      VARCHAR(15) NOT NULL DEFAULT 'pending'
                CHECK (status IN ('pending', 'approved', 'rejected')),
    decided_by  INT,
    decided_at  TIMESTAMP,
    CONSTRAINT leave_dates_order CHECK (end_date >= start_date),
    -- pending means undecided; approved/rejected need who and when
    CONSTRAINT leave_decision_consistent CHECK (
        (status = 'pending'  AND decided_by IS NULL     AND decided_at IS NULL)
     OR (status <> 'pending' AND decided_by IS NOT NULL AND decided_at IS NOT NULL)
    ),
    -- only a parent linked to this student can decide the request
    CONSTRAINT leave_decider_is_parent
        FOREIGN KEY (student_id, decided_by)
        REFERENCES student_parent (student_id, parent_id)
);
```

Above: `leave_requests` *with CHECK constraints and the composite foreign key* `leave_decider_is_parent`. *Below: the GiST exclusion constraint and an index, from the same file.*

Triggers

Trigger	What it does
`trg_students_role`, `trg_parents_role`, `trg_face_events_reviewer_role`, `trg_violations_resolver_role`	A linked account must have the right role (FT040)
`trg_attendance_validate`	Student belongs to the roll call's hostel (FT003); mark inside the window (FT004); present is never downgraded (FT005)
`trg_leave_guard`	A decided leave is final (FT010); an ended request cannot be approved (FT011)
`trg_entry_exit_validate`	Own hostel (FT021), time order (FT023), IN/OUT alternate (FT022); per-student advisory lock

Stored procedures and functions

Name	What it does
`mark_attendance`	Records a roll-call result; `ON CONFLICT DO NOTHING` makes simultaneous scans safe
`record_entry_exit`	Writes a gate IN/OUT row and, in the same transaction, a late_return or no_leave violation unless leave covers the date
`close_attendance_session`	Marks non-scanners absent and raises violations using two data-modifying CTEs; locks the row; safe to repeat
`decide_leave`	A linked parent decides a pending request once; row lock; overlap raises FT034
`register_student`, `delete_student`	Create a student with login, parents and consent in one transaction; refuse deletion when history exists
11 `report_*` functions	CTE, GROUP BY/HAVING, window SUM, RANK, LEAD, FILTER, width_bucket, subqueries, full-text

Roles and row-level security

`db/migrations/004_rbac.sql` turns the five application roles into PostgreSQL roles. Nothing is granted to PUBLIC. The app connects as `ft_app` (not a superuser, cannot bypass row-level security) and, for wrapped requests, runs `SET LOCAL ROLE ft_<role>` with the user id in one transaction; 26 policies then limit a student to their own rows and a parent to their children's. Password hashes are readable only by `ft_app`.

```
   tablename    | policyname  |  cmd   |                   roles
----------------+-------------+--------+-----------------------------------------
 attendance     | own_rows    | SELECT | {ft_student}
 attendance     | staff_all   | ALL    | {ft_admin,ft_app,ft_security,ft_warden}
 attendance     | ward_rows   | SELECT | {ft_parent}
 entry_exit_log | own_rows    | SELECT | {ft_student}
 entry_exit_log | staff_all   | ALL    | {ft_admin,ft_app,ft_security,ft_warden}
 entry_exit_log | ward_rows   | SELECT | {ft_parent}
 leave_requests | own_cancel  | DELETE | {ft_student}
 leave_requests | own_insert  | INSERT | {ft_student}
 leave_requests | own_rows    | SELECT | {ft_student}
 leave_requests | staff_all   | ALL    | {ft_admin,ft_app,ft_security,ft_warden}
 leave_requests | ward_decide | UPDATE | {ft_parent}
 leave_requests | ward_rows   | SELECT | {ft_parent}
 parents        | own_parents | SELECT | {ft_student}
 parents        | own_row     | SELECT | {ft_parent}
...
```

The full policy list (26 rows) is in section 7 of `07-advanced-sql.md` *and in* `db/schema.sql`.

6. SQL DML Statements

a) INSERT, UPDATE and DELETE from the real code paths

The statements the application or a stored function really runs, in one transaction that is rolled back (row counts before and after are printed). The database also refuses bad variants with its own error codes.

```
#### row counts before
SELECT (SELECT count(*) FROM users) users, (SELECT count(*) FROM students) students, (SELECT count(*) FROM leave_requests)
  leave, (SELECT count(*) FROM attendance) attendance, (SELECT count(*) FROM violations) violations;
 users | students | leave | attendance | violations
-------+----------+-------+------------+------------
    42 |       20 |    32 |        909 |        131
(1 row)
BEGIN;
BEGIN
#### DML 1. INSERT: register a student with one parent (function register_student, db/b_register.sql:35)
SELECT register_student(
  '{"email":"evidence.student@facetrack.test","hash":"PLACEHOLDER","roll_no":"24EV001","name":"Evidence
    Student","hostel_id":1,"room_no":"A-130"}'::jsonb,
  '[{"email":"evidence.parent@facetrack.test","hash":"PLACEHOLDER","name":"Evidence
    Parent","phone":"+919800000001","relation":"father"}]'::jsonb,
  true, 3) AS result
returned jsonb:
{"parents": [{"email": "evidence.parent@facetrack.test", "created": true, "user_id": 44, "parent_id": 20}], "user_id": 43,
  "student_id": 21}
SELECT s.student_id, s.roll_no, s.name, s.hostel_id, s.room_no, u.email, u.role FROM students s JOIN users u USING
  (user_id) WHERE s.roll_no='24EV001';
 student_id | roll_no |       name       | hostel_id | room_no |             email              |  role
------------+---------+------------------+-----------+---------+--------------------------------+---------
         21 | 24EV001 | Evidence Student |         1 | A-130   | evidence.student@facetrack.test | student
(1 row)
SELECT sp.student_id, sp.parent_id, sp.relation, p.name, p.phone FROM student_parent sp JOIN parents p USING (parent_id)
  JOIN students s USING (student_id) WHERE s.roll_no='24EV001';
 student_id | parent_id | relation |      name       |     phone
------------+-----------+----------+-----------------+---------------
         21 |        20 | father   | Evidence Parent | +919800000001
(1 row)
SELECT c.student_id, c.consented_at IS NOT NULL AS consent_recorded, c.recorded_by FROM b_consents c JOIN students s USING
  (student_id) WHERE s.roll_no='24EV001';
 student_id | consent_recorded | recorded_by
------------+------------------+-------------
         21 | t                |           3
(1 row)
SELECT student_id AS sid FROM students WHERE roll_no='24EV001'
SELECT parent_id AS pid FROM student_parent WHERE student_id=21
#### DML 1b. same function, duplicate SRN: the database refuses (FT051)
SAVEPOINT s1;
SAVEPOINT
SELECT register_student('{"email":"other@facetrack.test","hash":"PLACEHOLDER","roll_no":"24ev001","name":"Dup","hostel_id":
  1}'::jsonb,
  '[{"email":"p2@facetrack.test","hash":"PLACEHOLDER","name":"P","relation":"mother"}]'::jsonb, true, 3);
ERROR:  SRN already in use
CONTEXT:  PL/pgSQL function register_student(jsonb,jsonb,boolean,integer) line 30 at RAISE
ROLLBACK TO SAVEPOINT s1;
ROLLBACK
```

```
#### DML 3. INSERT then UPDATE: request leave (app/app/api/leave/route.ts:122), parent decides it (procedure decide_leave
  db/procedures.sql:236)
INSERT INTO leave_requests (student_id, start_date, end_date, reason)
VALUES (21, current_date + 3, current_date + 4, 'Family wedding out of town')
RETURNING leave_id, student_id, start_date, end_date, reason, status, decided_by, decided_at IS NULL AS undecided
INSERT 0 1
SELECT leave_id AS lid FROM leave_requests WHERE student_id=21 AND status='pending'
CALL decide_leave(33, 20, 'approved');
CALL
SELECT leave_id, status, decided_by = 20 AS decided_by_the_parent, decided_at IS NOT NULL AS has_decision_time FROM
  leave_requests WHERE leave_id=33;
 leave_id |  status  | decided_by_the_parent | has_decision_time
----------+----------+-----------------------+-------------------
       33 | approved | t                     | t
(1 row)
--- a decided request is final: a second decision and a direct UPDATE are both refused (FT032, FT010):
SAVEPOINT s3;
SAVEPOINT
CALL decide_leave(33, 20, 'rejected');
ERROR:  Leave request 33 was already approved
CONTEXT:  PL/pgSQL function decide_leave(integer,integer,text) line 19 at RAISE
ROLLBACK TO SAVEPOINT s3;
ROLLBACK
SAVEPOINT s4;
SAVEPOINT
UPDATE leave_requests SET status='pending', decided_by=NULL, decided_at=NULL WHERE leave_id=33;
ERROR:  Leave request 33 is already approved and cannot be changed
CONTEXT:  PL/pgSQL function fn_leave_guard() line 4 at RAISE
ROLLBACK TO SAVEPOINT s4;
ROLLBACK
#### everything above is rolled back
ROLLBACK;
ROLLBACK
#### row counts after (identical to before)
SELECT (SELECT count(*) FROM users) users, (SELECT count(*) FROM students) students, (SELECT count(*) FROM leave_requests)
  leave, (SELECT count(*) FROM attendance) attendance, (SELECT count(*) FROM violations) violations;
 users | students | leave | attendance | violations
-------+----------+-------+------------+------------
    42 |       20 |    32 |        909 |        131
(1 row)
```

b) Queries: joins, nested, correlated and recursive

Source file and lines are under each title; output is real `psql` output (first rows).

Join and CTE

Q4. Violations per hostel and type: three-table inner JOIN + GROUP BY

Source: db/queries.sql` lines 48-53

```
SELECT h.name AS hostel, v.type, count(*) AS total
FROM violations v
JOIN students s USING (student_id)
JOIN hostels  h ON h.hostel_id = s.hostel_id
GROUP BY h.name, v.type
ORDER BY h.name, total DESC, v.type;
```

```
          hostel          |       type        | total
--------------------------+-------------------+-------
 Aryabhatta Boys Hostel   | missed_attendance |    31
 Aryabhatta Boys Hostel   | late_return       |    20
 Aryabhatta Boys Hostel   | no_leave          |     4
 Kalam Girls Hostel       | missed_attendance |    31
 Kalam Girls Hostel       | late_return       |    24
 Kalam Girls Hostel       | no_leave          |     5
 Visvesvaraya Boys Hostel | missed_attendance |     9
 Visvesvaraya Boys Hostel | late_return       |     7
...
```

CTE 2. Two CTEs LEFT JOINed to students, hostels and a view

Source: db/b_queries.sql` lines 140-157

```
WITH open_v AS (
    SELECT student_id, count(*) AS open_violations
    FROM violations WHERE status = 'open' GROUP BY student_id
), missed AS (
    SELECT student_id, count(*) AS absences
    FROM attendance WHERE status = 'absent' GROUP BY student_id
)
SELECT s.name, h.name AS hostel,
       COALESCE(o.open_violations, 0) AS open_violations,
       COALESCE(m.absences, 0)        AS absences,
       p.presence
FROM students s
JOIN hostels h USING (hostel_id)
JOIN v_current_presence p USING (student_id)
LEFT JOIN open_v o USING (student_id)
LEFT JOIN missed m USING (student_id)
WHERE COALESCE(o.open_violations, 0) + COALESCE(m.absences, 0) > 0
ORDER BY open_violations DESC, absences DESC, s.name;
```

```
       name        |          hostel          | open_violations | absences | presence
-------------------+--------------------------+-----------------+----------+----------
 Riya Kulkarni     | Kalam Girls Hostel       |              11 |       16 | inside
 Aarav Sharma      | Aryabhatta Boys Hostel   |              10 |       15 | inside
 Ananya Krishnan   | Kalam Girls Hostel       |               6 |        5 | inside
 Rahul V           | Aryabhatta Boys Hostel   |               5 |       10 | inside
 Vihaan Reddy      | Visvesvaraya Boys Hostel |               4 |        8 | inside
 Meera Pillai      | Kalam Girls Hostel       |               3 |        8 | inside
 Karan Iyer        | Aryabhatta Boys Hostel   |               3 |        7 | inside
 Sneha M           | Kalam Girls Hostel       |               3 |        1 | inside
...
```

Nested query

Q6. IN subquery

Source: db/queries.sql` lines 69-74

```
SELECT roll_no, name
FROM students
WHERE student_id IN (SELECT student_id
                     FROM leave_requests
                     WHERE status = 'approved'
                       AND DATE '2026-10-24' BETWEEN start_date AND end_date);
```

```
 roll_no |    name
---------+------------
 21CS001 | Arun Kumar
(1 row)
```

Correlated subquery

Q5. Correlated scalar subquery: students whose latest gate event is OUT

Source: db/queries.sql` lines 57-65

```
-- Their most recent gate event is an OUT.
SELECT s.roll_no, s.name
FROM students s
WHERE (SELECT l.direction
       FROM entry_exit_log l
       WHERE l.student_id = s.student_id
       ORDER BY l.event_time DESC, l.log_id DESC
       LIMIT 1) = 'OUT'
ORDER BY s.roll_no;
```

```
 roll_no | name
---------+------
(0 rows)
```

Recursive query (view v_absence_runs, WITH RECURSIVE)

RECURSIVE 1. Longest run of consecutive absent days per student, using the view

Source: db/b_queries.sql` lines 230-239

```
SELECT s.roll_no, s.name, h.name AS hostel,
       max(r.days)                                          AS longest_run_days,
       (array_agg(r.run_start ORDER BY r.days DESC, r.run_start))[1] AS run_started,
       count(*)                                             AS runs
FROM v_absence_runs r
JOIN students s ON s.student_id = r.student_id
JOIN hostels  h ON h.hostel_id  = s.hostel_id
GROUP BY s.student_id, s.roll_no, s.name, h.name
ORDER BY longest_run_days DESC, s.roll_no
LIMIT 10;
```

```
 roll_no |      name       |          hostel          | longest_run_days | run_started | runs
---------+-----------------+--------------------------+------------------+-------------+------
 22CS104 | Aarav Sharma    | Aryabhatta Boys Hostel   |                5 | 2026-10-10  |    9
 21CS007 | Karthik R       | Aryabhatta Boys Hostel   |                4 | 2026-10-02  |    3
 23EC135 | Diya Venkatesh  | Kalam Girls Hostel       |                4 | 2026-10-18  |    3
 21CS001 | Arun Kumar      | Aryabhatta Boys Hostel   |                3 | 2026-10-14  |    3
 21CS014 | Divya S         | Kalam Girls Hostel       |                3 | 2026-10-01  |    3
 21EC012 | Rahul V         | Aryabhatta Boys Hostel   |                3 | 2026-10-13  |    7
 21EE118 | Siddharth Menon | Visvesvaraya Boys Hostel |                3 | 2026-09-28  |    4
 22CE164 | Riya Kulkarni   | Kalam Girls Hostel       |                3 | 2026-10-12  |   10
...
```

The recursive view `v_absence_runs` (`db/b_queries.sql` 205-226) is part of the project: the anchor selects the first absent day of each run and the recursive step extends it by the next consecutive day. It has a fixture test.

c) Test runs of procedures, triggers and roles

All SQL suites pass on fresh databases: `test_cases` 98, `b_register_tests` 26, `b_reports_tests` 16, `b_demo_tests` 10, `b_queries_tests` 6, `rbac_tests` 76 checks. Lines from the real output:

```
 33 | PASS | [mark_attendance] matched scan is accepted | ok | ok
 34 | PASS | [mark_attendance] stored status is present | present | present
 35 | PASS | [mark_attendance FT001] second scan of the same student | FT001 | FT001
 36 | PASS | [mark_attendance] low-confidence scan is accepted | ok | ok
 37 | PASS | [mark_attendance] stored status is low_confidence | low_confidence | low_confidence
 38 | PASS | [mark_attendance FT002] session does not exist | FT002 | FT002
...

 20 | PASS | [trigger FT003] attendance for a student of another hostel | FT003 | FT003
 21 | PASS | [trigger FT004] attendance mark outside the roll-call window | FT004 | FT004
 22 | PASS | [trigger FT005] present cannot be downgraded to absent | FT005 | FT005
 23 | PASS | [trigger] warden review: low_confidence becomes present | ok | ok
 24 | PASS | [trigger FT010] a decided leave request cannot be edited | FT010 | FT010
 25 | PASS | [trigger] create an old pending leave for the next test | ok | ok
...
 1 | ft_app | is neither superuser nor BYPASSRLS, cannot create roles or databases | f,f,f,f | false,false,false,false |
   PASS
 2 | ft_* | the five role roles cannot log in | 5 x NOLOGIN | 5 x NOLOGIN | PASS
 3 | PUBLIC | no table in public is readable by PUBLIC | 0 | 0 | PASS
 4 | ft_app | read a password hash for login | rows=1 | rows=1 | PASS
 5 | ft_app | record a login attempt | rows=1 | rows=1 | PASS
 6 | ft_app | cannot drop a table | denied | denied | PASS
 7 | ft_student | sees only their own student row | rows=1 | rows=1 | PASS
 8 | ft_student | sees only their own leave (3 of the 4 fixture rows belong to them) | rows=3 | rows=3 | PASS
...
```

d) Query performance: EXPLAIN before and after an index

`db/b_explain_demo.sql` on 300,000 synthetic gate rows (rolled back): a B-tree index turns a sequential scan into an index scan, and a GIN index does the same for full-text search.

```
######## DEMO 1: latest gate events of one student (B-tree on student_id, event_time DESC)
--- BEFORE: index dropped
         ->  Seq Scan on entry_exit_log  (cost=0.00..5661.06 rows=50421 width=15) (actual time=0.013..21.278 rows=50002
           loops=1)
 Execution Time: 28.596 ms
--- AFTER: index created
   ->  Index Scan using idx_log_student_time on entry_exit_log  (cost=0.42..9188.91 rows=50421 width=15) (actual
     time=0.044..0.046 rows=10 loops=1)
 Execution Time: 0.078 ms
######## DEMO 2: full-text search for a rare word in leave reasons (GIN)
--- BEFORE: GIN index dropped
 Seq Scan on leave_requests  (cost=0.00..2915.05 rows=500 width=27) (actual time=9.327..9.328 rows=1 loops=1)
 Execution Time: 9.336 ms
--- AFTER: GIN index created
 Bitmap Heap Scan on leave_requests  (cost=19.71..1098.93 rows=500 width=27) (actual time=0.009..0.010 rows=1 loops=1)
   ->  Bitmap Index Scan on idx_leave_reason_fts  (cost=0.00..19.59 rows=500 width=0) (actual time=0.005..0.005 rows=1
     loops=1)
 Execution Time: 0.021 ms
```

7. NoSQL / Vector Database Commands

Qdrant 1.19.2 holds the face embeddings: collection `face_embeddings`, 512 dimensions, Cosine distance, payload `{student_id, face_profile_id, active}`, integer index on `student_id`. The commands ran against a throw-away collection with **random placeholder vectors** (no real face or embedding appears); vectors are abbreviated.

Operation	Application code
Create collection, payload index; upsert 5 points per enrolment	`app/lib/qdrant.ts`, `app/lib/faces.ts enrolEmbeddings()`
Search nearest 10 (gate scan); filtered search (duplicate-face guard)	`app/lib/matching.ts`, `findDuplicateFace()`
Read one sample or all samples of a student (admin)	`GET /api/faces/[id]`, `/api/admin/students/[id]/faces`
Update payload `active` true/false (admin)	`PATCH /api/faces/[id]`; `setPayload`

Operation	Application code
Delete points by id (admin)	`DELETE /api/faces/[id]; deleteProfile()`

Link to PostgreSQL: `face_profiles` has one row per point (`qdrant_point_id` UUID UNIQUE, ON DELETE CASCADE from the student). No foreign key can cross two systems, so enrolment writes rows in a transaction, upserts the points, commits, and on failure rolls back and deletes the points.

Create the collection

```
PUT http://localhost:6336/collections/evidence_demo
{
  "vectors": {
    "size": 512,
    "distance": "Cosine"
  }
...
Response: {"result": true, "status": "ok", "time": 0.03229525}
```

Similarity search, nearest 3

```
POST http://localhost:6336/collections/evidence_demo/points/query
{
  "query": [0.022804, 0.015178, 0.050996, ... 512 numbers],
  "limit": 3,
  "with_payload": true
}
...
Response: {"result": {"points": [{"id": "11111111-1111-4111-8111-111111111111", "version": 3, "score": 0.99388975,
  "payload": {"student_id": 1, "face_profile_id": 100}}, {"id": "55555555-5555-4555-8555-555555555555", "version": 3,
  "score": 0.11594609, "payload": {"student_id": 2, "face_profile_id": 104}}, {"id":
  "44444444-4444-4444-8444-444444444444", "version": 3, "score": 0.018877575, "payload": {"student_id": 2, "face_profi
```

UPDATE a payload

```
POST http://localhost:6336/collections/evidence_demo/points/payload?wait=true
{
  "payload": {
    "face_profile_id": 999
  },
  "points": [
...
Response: {"result": {"operation_id": 4, "status": "completed"}, "status": "ok", "time": 0.000474833}
```

DELETE points by id

```
POST http://localhost:6336/collections/evidence_demo/points/delete?wait=true
{
  "points": [
    "22222222-2222-4222-8222-222222222222",
    "33333333-3333-4333-8333-333333333333"
  ]
...
Response: {"result": {"operation_id": 5, "status": "completed"}, "status": "ok", "time": 0.000306417}
```

The application's update route: deactivate one face sample (admin)

```
PATCH http://localhost:3400/api/faces/1
{"active": false}
Response (HTTP 200): {"sample": {"faceProfileId": 1, "studentId": 1, "qdrantPointId":
  "ef181694-e93c-488c-8e90-458b5df380db", "enrolledAt": "2026-11-03 10:11:16.127355", "active": false, "payload":
  {"student_id": 1, "face_profile_id": 1, "active": false}}}
```

Effect on matching (face_profile_id, score, active), without and with the filter the app uses

```
without the filter : [(1, 1.0, False), (3, 0.7262, True), (2, 0.7175, True)]
with the app's filter: [(3, 0.7262, True), (2, 0.7175, True), (11, 0.1394, True)]
```

8. Results

a) Resulting Tables' Screenshots

Database built by `./db/load_all.sh` plus the demo seed (14 generated students, 45 days of simulated history); all people and emails are fictional. Row counts of all 18 tables:

Table	Rows	Table	Rows	Table	Rows
users	42	students	20	parents	19
student_parent	21	hostels	3	gates	4
cameras	5	attendance_sessions	139	attendance	909
entry_exit_log	992	face_events	2011	face_profiles	18
leave_requests	32	violations	131	b_consents	14
login_attempts	2	b_meta	2	b_demo_rows	3165

students 20 rows in the scratch database; first 8 shown; password hashes and token versions are never selected

student_id	user_id	roll_no	name	hostel_id	room_no
1	4	21CS001	Arun Kumar	1	A-101
2	5	21CS007	Karthik R	1	A-102
3	6	21EC012	Rahul V	1	B-205
4	7	21CS014	Divya S	2	K-110
5	8	21IT003	Meena R	2	K-111
6	9	21EC020	Sneha M	2	K-204
7	16	22CS104	Aarav Sharma	1	A-201
8	18	22EC117	Vihaan Reddy	3	V-202

Figure 8.1: `students` *(20 rows, first 8 shown)*

attendance 909 rows in the scratch database; first 8 shown; password hashes and token versions are never selected

session_id	student_id	event_id	status	marked_at
139	8	1987	present	2026-11-02T20:43:47
139	10	1989	present	2026-11-02T20:51:03
139	12	1993	present	2026-11-02T20:38:26
139	14	1997	present	2026-11-02T20:31:24
138	4	1981	present	2026-11-02T20:34:16
138	5	1982	present	2026-11-02T20:57:34
138	6	1983	present	2026-11-02T20:37:23
138	15	2000	present	2026-11-02T20:55:29

Figure 8.2: `attendance` *(909 rows, first 8 shown)*

entry_exit_log 992 rows in the scratch database; first 8 shown; password hashes and token versions are never selected

log_id	student_id	gate_id	event_id	direction	event_time
992	20	3	2010	IN	2026-11-02T10:51:01
991	20	3	2009	OUT	2026-11-02T09:27:05
990	19	3	2007	IN	2026-11-02T11:31:11
989	19	3	2006	OUT	2026-11-02T08:52:05
988	17	3	2003	IN	2026-11-02T19:52:26
987	17	3	2002	OUT	2026-11-02T17:30:23
986	15	3	1999	IN	2026-11-02T17:20:56
985	15	3	1998	OUT	2026-11-02T14:30:31

Figure 8.3: `entry_exit_log` *(992 rows, first 8 shown)*

leave_requests 32 rows in the scratch database; first 8 shown; password hashes and token versions are never selected

leave_id	student_id	start_date	end_date	reason	status	decided_by	decided_at
32	5	2026-11-05	2026-11-06	Attending a cousin's convocation	pending	NULL	NULL
31	4	2026-11-05	2026-11-06	Attending a relative's housewarming	pending	NULL	NULL
30	3	2026-11-04	2026-11-05	Family pilgrimage trip	pending	NULL	NULL
29	2	2026-11-05	2026-11-07	Attending a relative's housewarming	pending	NULL	NULL
28	1	2026-11-05	2026-11-06	Temple festival in our village	pending	NULL	NULL
27	20	2026-09-29	2026-09-30	Pongal celebrations with family	approved	19	2026-09-28T12:48:00
26	20	2026-10-12	2026-10-14	Pongal celebrations with family	approved	19	2026-10-11T18:52:00
25	18	2026-09-23	2026-09-23	Fever, going home to recover	approved	14	2026-09-21T12:08:00

Figure 8.4: `leave_requests` *(32 rows, first 8 shown)*

violations 131 rows in the scratch database; first 8 shown; password hashes and token versions are never selected

violation_id	student_id	log_id	session_id	type	details	raised_at	status	resolved_by	resolved_at
131	20	NULL	138	missed_attendance	Absent at the roll call on 02 Nov 2026	2026-11-02T21:00:00	open	NULL	NULL
130	8	NULL	136	missed_attendance	Absent at the roll call on 01 Nov 2026	2026-11-01T21:00:00	open	NULL	NULL
129	17	NULL	135	missed_attendance	Absent at the roll call on 01 Nov 2026	2026-11-01T21:00:00	open	NULL	NULL
128	15	NULL	135	missed_attendance	Absent at the roll call on 01 Nov 2026	2026-11-01T21:00:00	open	NULL	NULL
127	15	NULL	132	missed_attendance	Absent at the roll call on 31 Oct 2026	2026-10-31T21:00:00	open	NULL	NULL
126	3	NULL	131	missed_attendance	Absent at the roll call on 31 Oct 2026	2026-10-31T21:00:00	open	NULL	NULL
125	15	NULL	129	missed_attendance	Absent at the roll call on 30 Oct 2026	2026-10-30T21:00:00	open	NULL	NULL
124	7	NULL	128	missed_attendance	Absent at the roll call on 30 Oct 2026	2026-10-30T21:00:00	open	NULL	NULL

Figure 8.5: `violations` *(131 rows, first 8 shown)*

face_profiles 18 rows in the scratch database; first 8 shown; password hashes and token versions are never selected

profile_id	student_id	qdrant_point_id	enrolled_at
1	1	ef181694-e93c-488c-8e90-458b5df380db	2026-11-03T10:11:16.127355
2	1	bd07b96a-58cf-4965-8664-e66575fe7eee	2026-11-03T10:11:16.127355
3	1	3d059666-3989-4770-b017-9e71477f542d	2026-11-03T10:11:16.127355
4	2	1d1ceeed-5c1b-421c-81a0-b3f512901593	2026-11-03T10:11:16.146752
5	2	ce5f882f-86e4-4ff6-a691-4c7cce45b1e9	2026-11-03T10:11:16.146752
6	2	9e4004b2-31b0-4836-8ddd-b03feb65c427	2026-11-03T10:11:16.146752
7	3	3f1570d6-6fee-473d-b570-81d3d0107ddc	2026-11-03T10:11:16.172254
8	3	5ec9ea48-ae32-447e-8576-135ade3ee446	2026-11-03T10:11:16.172254

Figure 8.6: `face_profiles` *(18 rows, first 8 shown)*

9. Next.js Project Tree Structure

Tracked files only (no `node_modules`, `.next`, `.env` or images; faces are never stored in the repository). Shortened to folders and main files; # marks annotations.

```
.
|-- .gitignore
|-- .mailmap
|-- AUDIT2.md
|-- README.md
|-- app/    # Next.js 16 application (pages, API routes, libraries, scripts)
|   |-- .gitignore
|   |-- AGENTS.md
|   |-- README.md
|   |-- app/    # App Router: pages and API routes
|   |-- components/    # React components (UI kit, camera, report charts)
|   |-- eslint.config.mjs
|   |-- lib/    # server-side libraries: auth, db (pool, withRole), matching, qdrant, reports
|   |-- next.config.ts
|   |-- package-lock.json
|   |-- package.json
|   |-- scripts/    # CLI scripts: collection admin, demo seed/reset, test suites
|   `-- tsconfig.json
|-- db/    # PostgreSQL: schema, triggers, procedures, seed, reports, roles/RLS, tests, migrations
|   |-- b_demo.sql
|   |-- b_demo_tests.sql
|   |-- b_explain_demo.sql
|   |-- b_queries.sql    # views, CTEs, window functions, recursive view v_absence_runs
|   |-- b_queries_tests.sql    # fixture test of the recursive view (6 checks, new)
|   |-- b_register.sql
|   |-- b_register_tests.sql
|   |-- b_reports.sql
|   |-- b_reports_tests.sql
|   |-- b_search.sql
|   |-- ddl_demo.sql
|   |-- demo_passwords.sql
|   |-- docs/    # phase design notes and the ER diagram (png, svg, dbml)
|   |-- explain_demo.sql
|   |-- load_all.ps1
|   |-- load_all.sh
|   |-- load_b.sh
|   |-- migrate.sh
|   |-- migrations/    # idempotent upgrade scripts (004 = roles, grants, row-level security)
|   |-- procedures.sql    # 4 procedures, login rate-limit functions
|   |-- queries.sql
|   |-- rbac_tests.sql    # per-role allowed/denied and row-level rules (76 checks, new)
|   |-- reports.sql
|   |-- schema.sql    # tables, constraints, indexes, and (section 6) roles/grants/RLS
|   |-- seed.sql
|   |-- settings.sql
|   |-- test_cases.sql
|   |-- transactions_demo.sql
|   `-- triggers.sql    # 7 triggers and the exclusion constraint
|-- docker-compose.yml
|-- docs/    # architecture, ER model, vector design, privacy policy, test results
|   |-- architecture-components.svg
|   |-- architecture-gate-scan.svg
|   |-- architecture.md
|   |-- er-model.md
|   |-- important-queries.md
|   |-- privacy-and-biometric-policy.md
|   |-- query-optimization.md
|   |-- test-results.md
|   `-- vector-design.md
`-- embedder/    # Python FastAPI face-embedding service (InsightFace)
```

10. Sample Screenshots of Application Code and Demo

a) Application code samples

Gate rules in the database (`db/triggers.sql` 165-205)

db/triggers.sql lines 165–205

```
165  CREATE OR REPLACE FUNCTION fn_entry_exit_validate() RETURNS trigger
166  LANGUAGE plpgsql AS $$
167  DECLARE
168      v_student_hostel INT;
169      v_gate_hostel    INT;
170      v_last           entry_exit_log%ROWTYPE;
171  BEGIN
172      PERFORM pg_advisory_xact_lock(7001, NEW.student_id);
173
174      SELECT hostel_id INTO v_student_hostel FROM students WHERE student_id = NEW.student_id;
175      SELECT hostel_id INTO v_gate_hostel    FROM gates    WHERE gate_id    = NEW.gate_id;
176
177      IF v_student_hostel IS DISTINCT FROM v_gate_hostel THEN
178          RAISE EXCEPTION 'Gate % does not belong to the hostel of student %',
179              NEW.gate_id, NEW.student_id USING ERRCODE = 'FT021';
180      END IF;
181
182      SELECT * INTO v_last
183      FROM entry_exit_log
184      WHERE student_id = NEW.student_id
185      ORDER BY event_time DESC, log_id DESC
186      LIMIT 1;
187
188      IF FOUND THEN
189          IF NEW.event_time <= v_last.event_time THEN
190              RAISE EXCEPTION 'Event at % is not after the last gate event of student % (%)',
191                  NEW.event_time, NEW.student_id, v_last.event_time USING ERRCODE = 'FT023';
192          END IF;
193          IF NEW.direction = v_last.direction THEN
194              RAISE EXCEPTION 'Student % is already marked % (since %)',
195                  NEW.student_id, v_last.direction, v_last.event_time USING ERRCODE = 'FT022';
196          END IF;
197      END IF;
198
199      RETURN NEW;
200  END $$;
201
202  DROP TRIGGER IF EXISTS trg_entry_exit_validate ON entry_exit_log;
203  CREATE TRIGGER trg_entry_exit_validate
204      BEFORE INSERT ON entry_exit_log
205      FOR EACH ROW EXECUTE FUNCTION fn_entry_exit_validate();
```

Role check and database role in an API route (`app/app/api/leave/[id]/decide/route.ts` 8-43)

app/app/api/leave/[id]/decide/route.ts lines 8–43

```
 8  export async function POST(
 9    request: Request,
10    { params }: { params: Promise<{ id: string }> }
11  ) {
12    try {
13      const user = await requireRole(["parent"]);
14      return await withRole(user, async (db) => {
15        if (!user.parentId) {
16          return Response.json({ error: "Your account is not linked to a parent record" }, { status: 403 });
17        }
18
19        const leaveId = parseId((await params).id);
20        if (leaveId === null) {
21          return Response.json({ error: "Invalid leave request id" }, { status: 400 });
22        }
23
24        let body: unknown;
25        try {
26          body = await request.json();
27        } catch {
28          return Response.json({ error: 'Send JSON like {"decision":"approved"}' }, { status: 400 });
29        }
30        const decision = (body as { decision?: unknown } | null)?.decision;
31        if (decision !== "approved" && decision !== "rejected") {
32          return Response.json({ error: "decision must be approved or rejected" }, { status: 400 });
33        }
34
35        await db.query("CALL decide_leave($1, $2, $3)", [leaveId, user.parentId, decision]);
36        return Response.json({ ok: true, id: leaveId, status: decision });
37      });
38    } catch (e) {
39      const r = authErrorResponse(e) ?? dbErrorResponse(e);
40      if (r) return r;
41      throw e;
42    }
43  }
```

b) Authentication and role-based access control

Front end. The login form posts to `POST /api/login` and opens the home page of the role. The token lives in an HttpOnly cookie, never in page JavaScript. **Back end.** Login checks the input, asks `login_retry_after`

whether the email or address is locked (5 failures per email or 20 per address in 15 minutes gives HTTP 429), compares the password with bcrypt (always one comparison, with one generic error), and signs a JWT (HS256, 8 hours) carrying the role and a token version. Every protected request goes through `requireRole(roles)`, which verifies the cookie, re-reads the role and `token_version` from the database (a changed role or password change revokes older tokens) and answers 401 or 403.

Routes and pages	Allowed roles
Leave request and cancel	student (own pending requests)
Leave and violation lists	student (own), parent (wards), warden, admin
Leave decision	parent of that student
Resolve violations, roll calls, review queue, reports, `/warden/*`	warden, admin
Gate scan	security, admin
Registration, enrolment, face samples, sign out everywhere	admin

In the database. The five roles also exist as PostgreSQL roles; `withRole` runs a request as `ft_student` or `ft_parent` with row-level security (76 tests pass). Only the leave, violation, history and wards routes use it; the other routes run as the app login `ft_app`, which holds the staff roles together, so `requireRole` is their only guard (see section 11).

c) Application demo screenshots

Figure 10.1 /login (left) and 10.2 /admin/students/new (right)

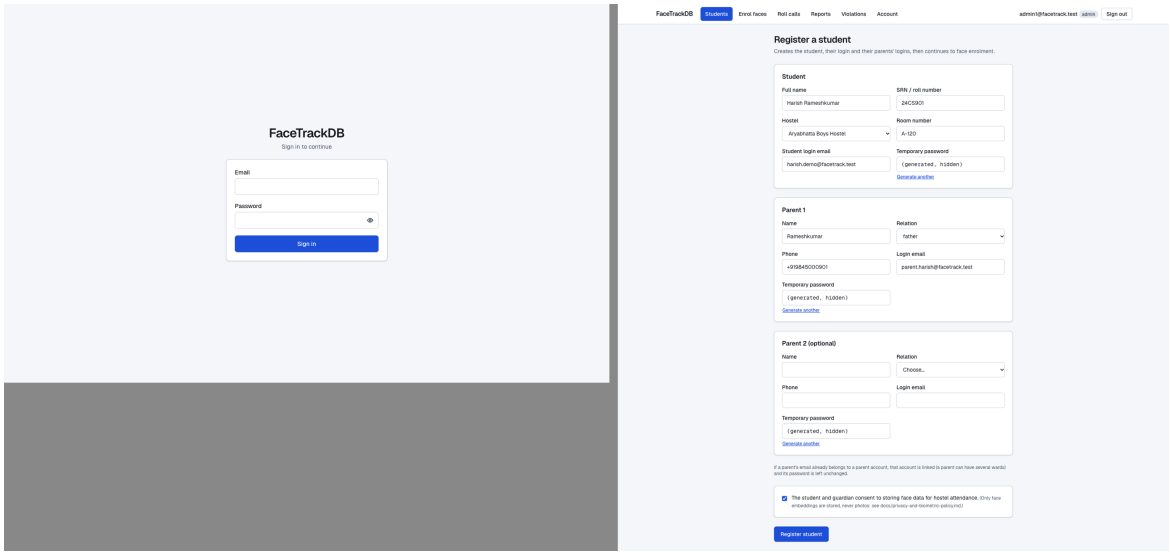


Figure 10.3: /enrol (admin): camera view, face guide, 5-frame progress

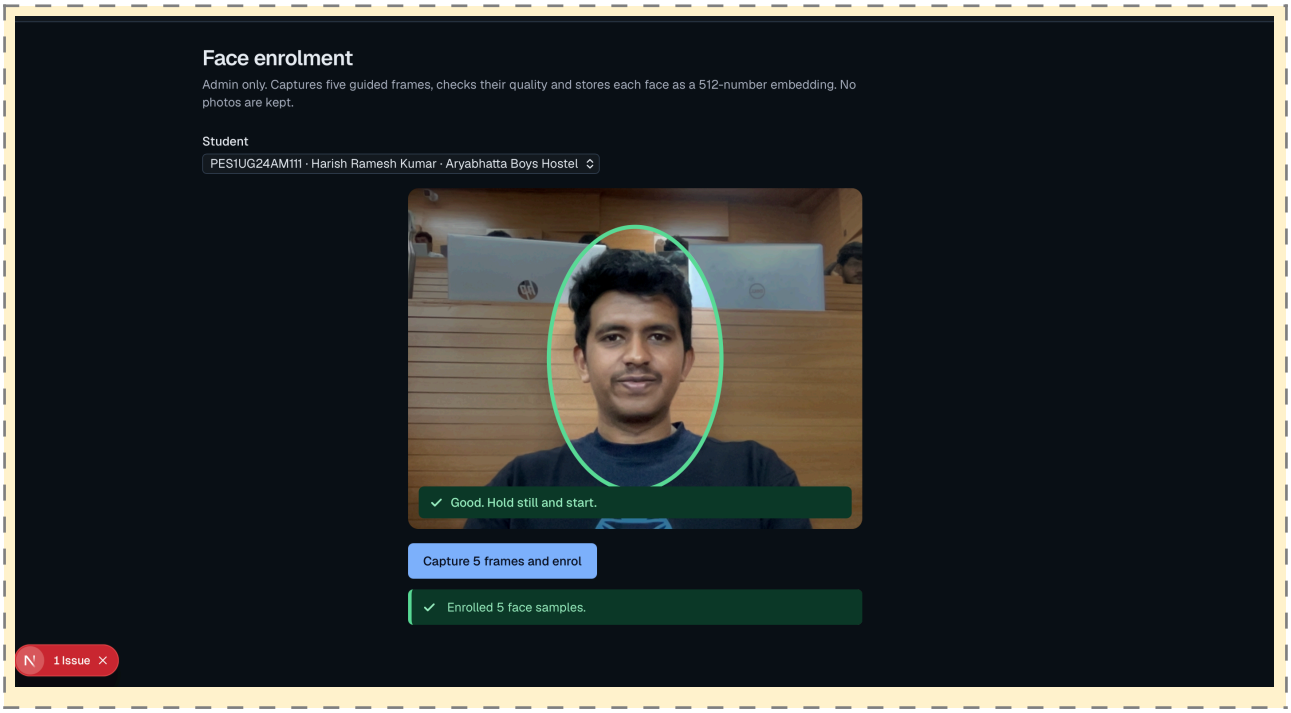


Figure 10.4: /gate (security): "Marked present" for the enrolled face

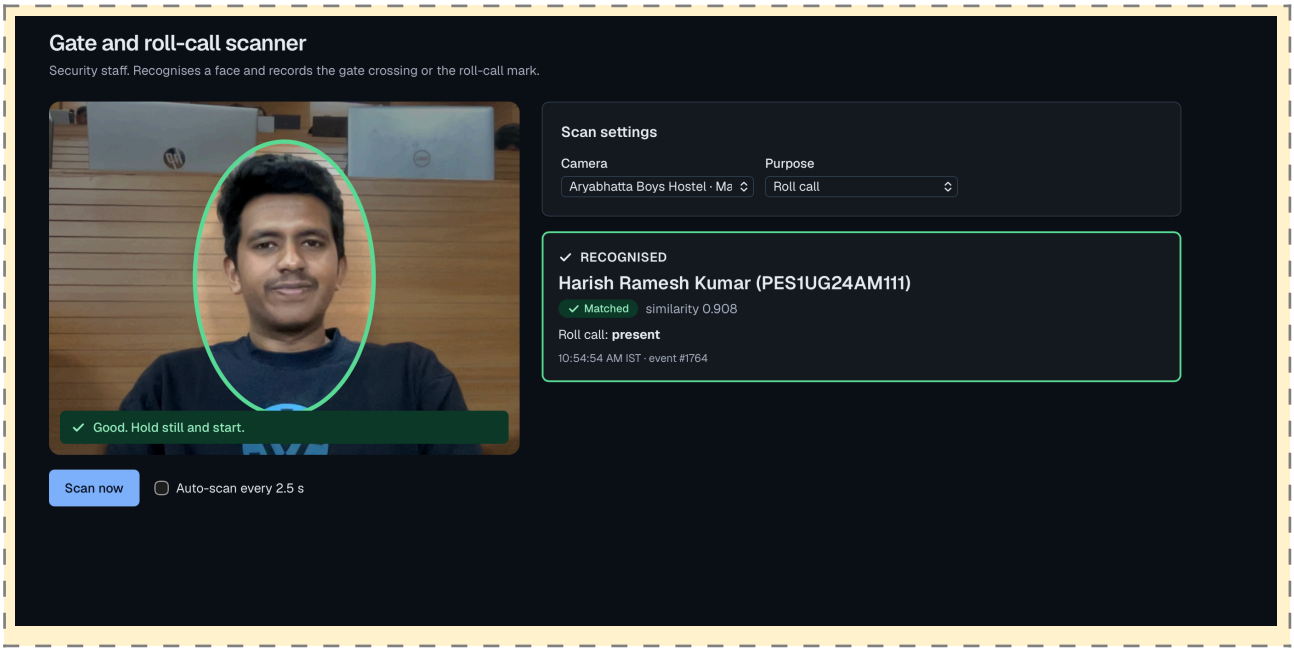


Figure 10.5: /gate (security): "No match" for a stranger

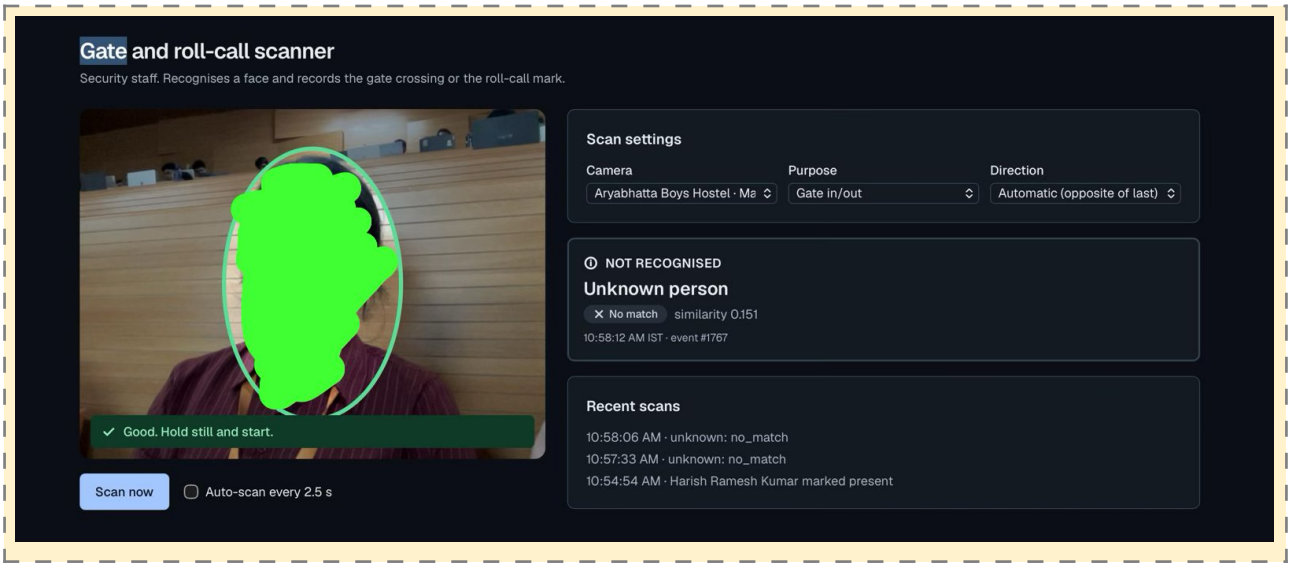


Figure 10.6 /warden (left) and 10.7 /warden/search?q=curfew (right)

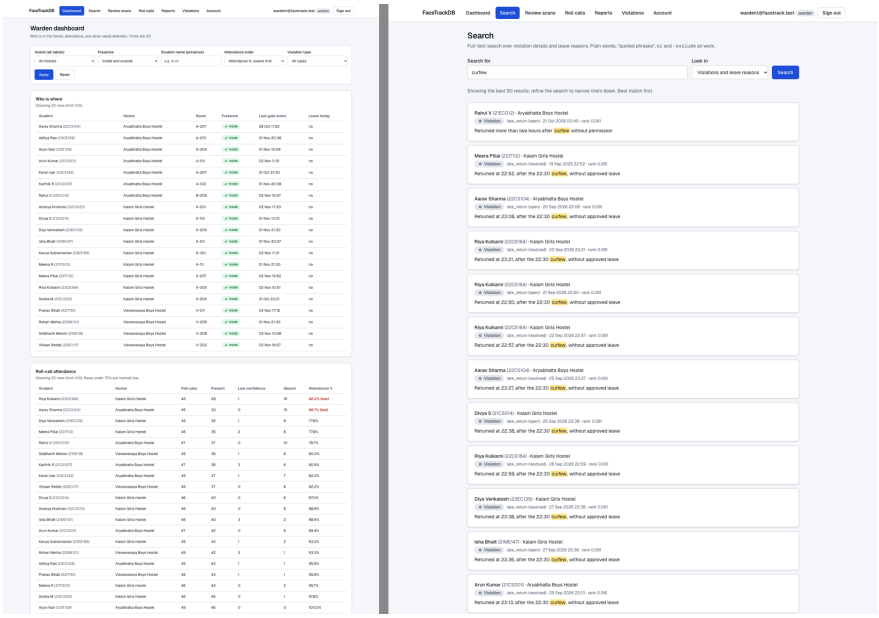


Figure 10.8 /warden/review (left) and 10.9 /warden/sessions (right)

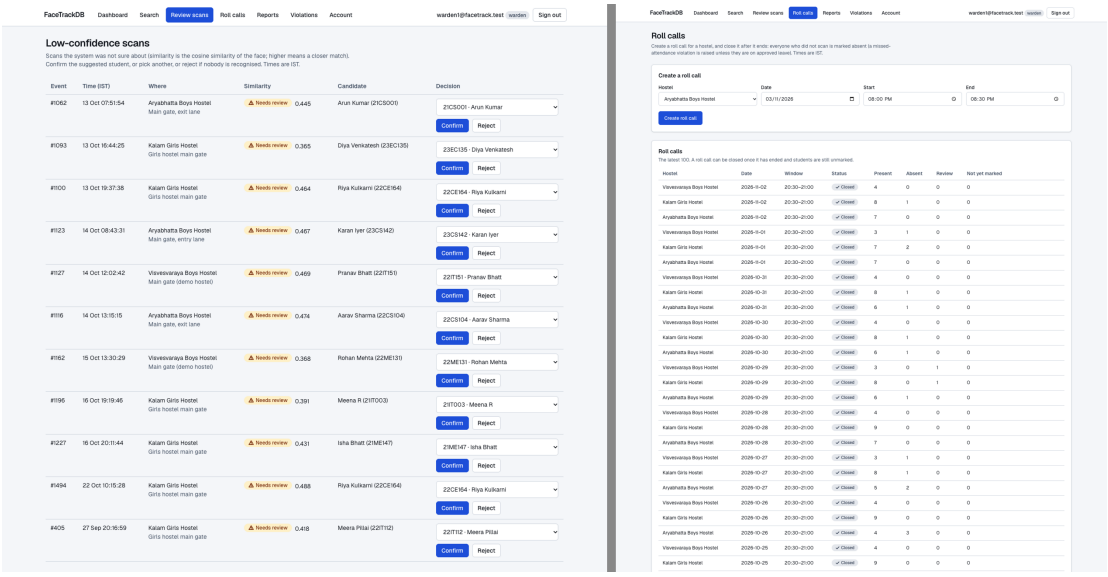


Figure 10.10 /student/leave (left) and 10.11 /parent/leave (right)

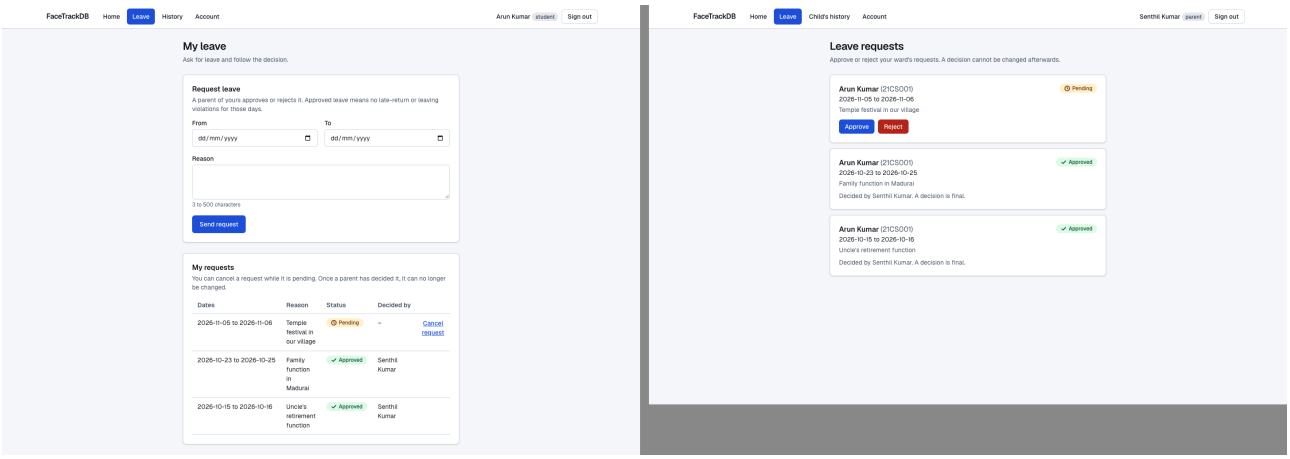


Figure 10.12 /reports/present (warden): slides 1 to 4 of 11



11. Evaluation Rubrics - Completion Status

#	Criteria	Distribution	Status	Evidence and notes
1	Database Design	ER diagram, relational schema with keys, relationships and constraints	Completed	Sections 3 and 4; diagrams regenerated from the live database (18 tables, 26 foreign keys).
2	Database Creation	SQL database implementation: DDL with constraints; NoSQL/Vector database creation; technology justification	Completed	Sections 2, 5 and 7; one command builds the database; images pinned.
3	SQL Operations and Queries	DML operations, nested/ correlated/recursive queries, join queries	Completed	Section 6, including the recursive view `v_absence_runs`.
4	Advanced Database Concepts	Procedure/function, trigger, CTE; NoSQL/Vector feature read, update, delete	Completed	Sections 5-7: 4 procedures, 26 functions, 7 triggers, CTEs (also recursive), full-text search, roles with row-level security; Qdrant read, update, delete as app routes.
5	Web Application and Database Integration	Next.js interface; backend/API and database integration	Completed	Sections 9 and 10: 23 pages and 23 API route files over PostgreSQL and Qdrant.
6	Authentication and Authorization	Authentication (front end and backend), minimum two roles, role-based access control in database	Completed, coverage partial	Section 10b: login, five roles, PostgreSQL roles and 26 RLS policies (76 tests). Only the leave, violation, history and wards routes run through the database roles; the other routes are guarded by the app alone.
7	Documentation, Git/ GitHub, Demonstration, Testing, Originality	Documentation, Git/GitHub, originality and individual contribution	Completed	README, architecture, ER model, vector design, privacy and test-results documents; GitHub repository; demo script; tests below.

Testing summary

SQL suites (fresh databases): `test_cases` 98/98, `b_register_tests` 26/26, `b_reports_tests` 16/16, `b_demo_tests` 10/10, `b_queries_tests` 6/6, `rbac_tests` 76/76. Application suites: `test:match` 38 and 35 (cooldown 3 and 0), `test:register` 38, `test:faces` 13, `test:reports` 14, `test:hardening` 30, `test:demo` 42, all passing. Suites run only against scratch databases.

Notes and limitations

- **Database-level access control is partial:** staff routes (dashboard, reports, roll calls, review, gate scan, registration, enrolment) run as the application login role, which holds the three staff roles together.
- **Recognition:** thresholds (0.50 match, 0.35-0.50 review, margin 0.08) are starting values checked with a few people; there is no liveness check; the camera pages were tested by hand only.
- **Simulated history:** 45 days of attendance, gate events, leave and violations were generated by `npm run demo:seed` and are labelled as generated; live scans are real records.
- **Other:** no global authorization middleware (a route that forgets `requireRole` would be public), no CSRF token, and the InsightFace models are for non-commercial use.

Division of work

Harshit Vaish (PES1UG24AM116): core schema, rules in the database (constraints, triggers, procedures), seed data, SQL tests, login, leave and violation features. **Harish Ramesh Kumar (PES1UG24AM111):** vector database and face matching, registration and enrolment, reporting SQL and presentation mode, dashboard, search and review pages, security hardening, database roles and row-level security, face-sample routes, recursive view,

user interface, demo data. Both reviewed and tested shared parts. GitHub: https://github.com/harish-rameshkumar/FaceTrackDB